Adaptation Experiment 2: Retrieval Baselines

1 Theoretical overview and goal

Let $(x_c^{(j)}, y_c^{(j)})_{j=1}^k$ be past windows retrieved for query x. After transferring each neighbor to the query scale, distance weights are

$$w_j = \frac{\exp(-\tilde{d}_j)}{\sum_{r=1}^k \exp(-\tilde{d}_r)}, \qquad \tilde{d}_j = \frac{d_j - \min_r d_r}{\mathrm{sd}(d) + \varepsilon}.$$

The simplest memory forecast is $\hat{y}_K = \sum_j w_j y_c^{(j)}$. If the frozen backbone's neighbor residual is $e_c^{(j)} = y_c^{(j)} - f(x_c^{(j)})$, residual adaptation instead uses $\hat{y}_R = \hat{y}_0 + \sum_j w_j e_c^{(j)}$. Scalar mixtures and ridge regressions learn how strongly these signals should correct the vanilla forecast.

Experimental goal. Determine whether retrieval contains useful predictive information before introducing learned gates or the TS-IFA neural adapter, and identify whether neighbor horizons or neighbor residuals are the more effective memory.

2 Implementation details

Shared extraction uses T_0 as the datastore, T_1 to fit baseline coefficients, T_2 for oracle diagnostics, and untouched T_3 for final evaluation. Online retrieval is period-aligned: the store grows with time but uses only eligible history. The current sweep uses Euclidean distance, raw and instance-normalized spaces, $k \in \{1, 3, 10\}$, daily period/stride 24, and at most 30,000 store windows.

The publication array adds ETTh1/2, ETTm1/2, Weather, Electricity, and Exchange with Chronos and TabPFN-TS; 572–64 is retained for Cross-RAG. TS-ICL is deferred. Atomic manifests prevent consumers from loading partial payloads.

Family	Compared predictions
Direct	Vanilla backbone; context-conditioned backbone
Neighbor	Weighted/mean neighbor horizon; weighted neighbor residual
Scalar fit	Convex horizon mixture; scaled residual correction
Ridge fit	Shared or horizon-specific residual, horizon, and full mixtures

Ridge features are RMS-standardized without centering, and coefficients use $\ell_2 = 10^{-3}$.

Sources: `src/experiments/extraction.py`; `src/adaptors/baselines/evaluate.py`
Extraction: `extract.slurm`
Baselines: `baselines.slurm`

The launcher also requests `-fit-baselines-on-eval`. Methods suffixed `eval_fit` are explicitly optimistic T_3 in-sample fits: they are diagnostic upper bounds and must not be reported as causal test results. The primary table should use coefficients fitted before T_3 and report MSE, nMSE, equal-user metrics, and W10 where available.

The baseline array depends on successful extraction. Resource lookup checks project-local and shared-parent candidates; moved copies set `DATA_ROOT` and `WEIGHTS_ROOT`. Tables report per-backbone details and equal-configuration means.

3 Interpretation

If residual retrieval wins, the memory primarily estimates backbone bias. If neighbor horizons win, local analog forecasting is sufficient. A large gap between valid fits and `eval_fit` indicates non-stationarity in the best mixture and motivates adaptive gating.